```
!pip install transformers datasets accelerate -q

import torch, math
from transformers import (
    GPT2LMHeadModel, GPT2Tokenizer, Trainer,
    TrainingArguments, DataCollatorForLanguageModeling, set_seed
)
from datasets import Dataset

set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2').to(device)

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Loading weights: 100%                                    148/148 [00:00<00:00, 4922.81it/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
```

```
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)

    with torch.no_grad():
        outputs = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
review_prompts = [
    "This product is",
    "I bought this phone and",
    "The quality of this item"
]

print("=== BASELINE OUTPUT ===")
baseline = {}

for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}")
    print(f"Output: {baseline[p]}")
```

```
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, yo
=== BASELINE OUTPUT ===

Prompt: This product is
Output: This product is designed to be used in conjunction with any other device and is intended for use in conjunction with

The following devices may be inserted into any such device, even if the device does not have a user interface or an open dev

a) any personal computer;

b) other personal computer or computer-based mobile device;

c) any computer-based home computer; and

d) any personal computer-based network-

Prompt: I bought this phone and
Output: I bought this phone and started working on it. I'm quite pleased with the way it performs, but this phone has the sa

Prompt: The quality of this item
Output: The quality of this item is the same as that of any other item on the item page.
```

```
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments and coding projects perfectly.",
    "the sound quality of these headphones is incredible with deep bass and clear vocals.",
    "this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.",
    "great wireless earbuds with noise cancellation that blocks out all background sound.",
    "the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.",
    "this portable charger saved me during travel and it charges my phone three times on a single charge.",
    "the tablet screen is bright and colorful which makes watching movies a great experience.",
    "i love this fitness tracker because it motivates me to reach my daily exercise goals.",
    "this bluetooth speaker is compact but delivers surprisingly loud and clear audio.",
    "the delivery was fast and the product was packed securely with no damage at all.",
    "excellent value for money and the build quality feels premium despite the affordable price.",
    "the customer service team was very helpful when i had questions about the product features.",
    "this camera takes stunning photos in low light and the video recording quality is very smooth.",
    "i have been using this product for three months and it still works perfectly like day one.",
    "the design is sleek and modern and it looks great on my desk next to my other gadgets.",
    "easy to set up right out of the box and the instructions were clear and simple to follow.",
    "highly recommend this product to anyone looking for quality and reliability at a fair price.",
    "the software updates keep adding new features which makes this purchase even more worthwhile.",
    "best purchase i made this year and i would definitely buy from this brand again."
]

dataset = Dataset.from_dict({"text": corpus})
```

```
def tokenize_function(example):
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

tokenized = dataset.map(tokenize_function, batched=True, remove_columns=["text"])
split = tokenized.train_test_split(test_size=0.15)
```

```
Map: 100%                                                    20/20 [00:00<00:00, 370.64 examples/s]
```

```
!pip install -U transformers
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./gpt2-reviews",
    num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    weight_decay=0.01,
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available()

)
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.5.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (0
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->h
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->t
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=data_collator
)

trainer.train()
```

```
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
```

[90/90 00:10, Epoch 10/10]

Step	Training Loss
10	3.811784
20	2.941562
30	1.906420
40	1.215388
50	1.256534
60	0.719734
70	0.621683
80	0.513918
90	0.526318

```
TrainOutput(global_step=90, training_loss=1.5014823224809435, metrics={'train_runtime': 11.4993,
'train_samples_per_second': 14.783, 'train_steps_per_second': 7.827, 'total_flos': 11104911360000.0, 'train_loss':
```

```
print("\n=== AFTER FINE-TUNING ===")
for p in review_prompts:
    out = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}")
    print(f"Before: {baseline[p][:100]}")
    print(f"After:  {out[:100]}")
```

```
=== AFTER FINE-TUNING ===

Prompt: This product is
Before: This product is designed to be used in conjunction with any other device and is intended for use in
After:  This product is packed securely with no damage at all. I recommend this product to anyone looking fo

Prompt: I bought this phone and
Before: I bought this phone and started working on it. I'm quite pleased with the way it performs, but this
After:  I bought this phone and it handles all my everyday tasks perfectly. The camera takes stunning photos

Prompt: The quality of this item
Before: The quality of this item is the same as that of any other item on the item page.
After:  The quality of this item is outstanding for the price and the product is packed securely with little
```

```
tokenizer2 = GPT2Tokenizer.from_pretrained('gpt2')
model2 = GPT2LMHeadModel.from_pretrained('gpt2').to(device)

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id
```

```
Loading weights: 100%                                    148/148 [00:00<00:00, 3505.28it/s]
```

```
recipes = [
    "to make butter chicken start by marinating chicken pieces in yogurt with turmeric chili powder and garam masala for or
    "heat butter in a pan and fry onions until golden brown then add ginger garlic paste and cook for two minutes.",
    "add tomato puree and cook on low heat for ten minutes until the oil separates from the masala.",
    "add the marinated chicken and cook on medium heat for fifteen minutes until fully cooked.",
    "finish with fresh cream and kasuri methi and serve hot with naan or steamed rice.",
    "for pasta carbonara boil spaghetti in salted water until al dente and reserve half cup of pasta water.",
    "fry diced pancetta in olive oil until crispy and set aside.",
    "whisk together egg yolks parmesan cheese and black pepper in a bowl.",
    "toss the hot pasta with pancetta and remove from heat then quickly stir in the egg mixture.",
    "the residual heat will cook the eggs into a creamy sauce and serve immediately with extra parmesan."
]

dataset2 = Dataset.from_dict({"text": recipes})

tokenized2 = dataset2.map(
```

```
    lambda x: tokenizer2(x["text"], truncation=True, padding="max_length", max_length=128),
    batched=True,
    remove_columns=["text"]
)

split2 = tokenized2.train_test_split(test_size=0.15)
```

Map: 100% 10/10 [00:00<00:00, 391.00 examples/s]

```
args2 = TrainingArguments(
    output_dir="./gpt2-recipes",
    num_train_epochs=10,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    weight_decay=0.01,
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available()
)
```

```
trainer2 = Trainer(
    model=model2,
    args=training_args,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)
)

trainer2.train()
```

[40/40 00:02, Epoch 10/10]

Step	Training Loss
10	3.538309
20	2.174660
30	1.417975
40	1.148893

```
TrainOutput(global_step=40, training_loss=2.0699591875076293, metrics={'train_runtime': 2.9576, 'train_samples_per_second':
27.049, 'train_steps_per_second': 13.525, 'total_flos': 5225840640000.0, 'train_loss': 2.0699591875076293, 'epoch': 10.0})
```

```
recipe_prompts = [
    "To make butter chicken",
    "For pasta carbonara",
    "To prepare chocolate cake"
]

print("\n=== RECIPE OUTPUT ===")

for p in recipe_prompts:
    print(f"\nPrompt: {p}")
    print(generate_text(model2, tokenizer2, p, max_length=150))
```

```
=== RECIPE OUTPUT ===

Prompt: To make butter chicken
To make butter chicken stock in a bowl add chicken stock and masala and cook for ten minutes. Add garam masala and cook for

Prompt: For pasta carbonara
For pasta carbonara is seasoned with parmesan and garam masala and served warm with turmeric garlic and parmesan.

This dish is easy to make with parmesan. It is also vegan and gluten-free. This is a delicious vegan parmesan with ginger ar

This post contains information on how to make parmesan with garlic and parmesan.

What is parmesan?

Parmesan is an Indian masala dish that is known as kasuri masala, a traditional Indian dish. However, parmesan has a very nu

This recipe can also be made from kasuri

Prompt: To prepare chocolate cake
To prepare chocolate cake for serving:


Mix together the egg yolks parmesan cheese and egg yolks parmesan into a paste and add to a bowl. Cook for 15 minutes until


Meanwhile, heat oil in a pan and fry onions until golden brown. Add garam masala and cook for few minutes until golden golde
```

```
Dosa masala and garam masala are available at
```